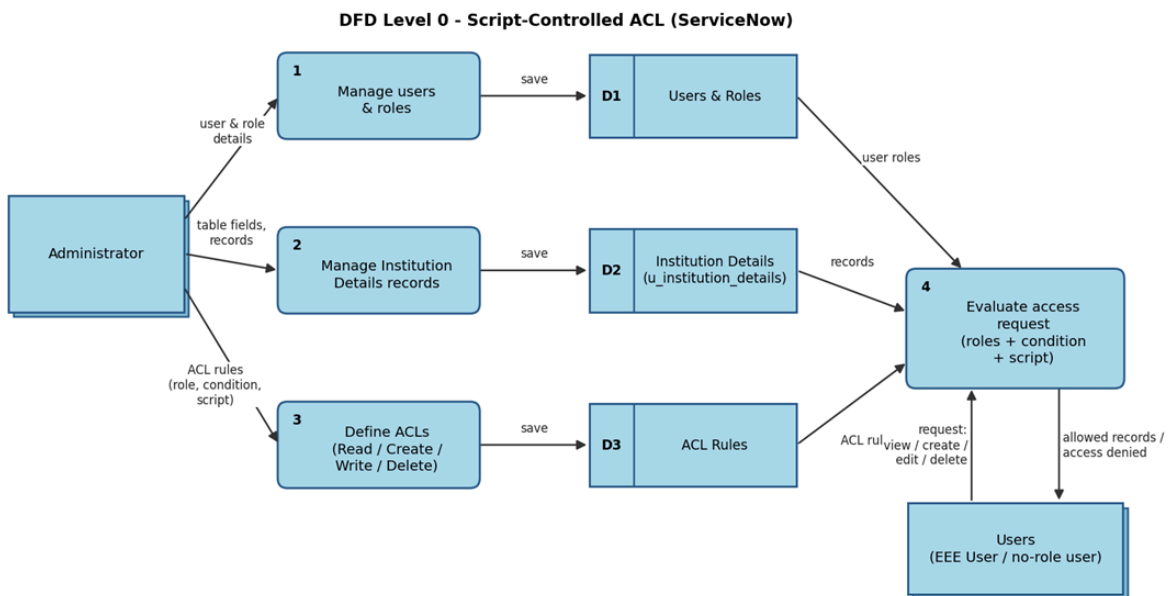


Project Design Phase-II Data Flow Diagram & User Stories

Date	30 september 2026
Team ID	SWTID-2026-2683
Project Name	Script-Controlled ACL – Restrict Record Access Based on Field Value (ServiceNow)
Maximum Marks	4 Marks

Data Flow Diagrams:

A Data Flow Diagram (DFD) shows how data enters and leaves the system, what changes the information, and where data is stored. For this project, the data flows between the administrator, the users, the ACL processing and the ServiceNow data stores.



Flow:

1. The administrator creates users and roles (bb1 to bb4), which are stored in Users & Roles.
2. The administrator creates the Institution Details table and its records, which are stored in Institution Details.
3. The administrator defines the READ, CREATE, WRITE and DELETE ACLs, which are stored as ACL Rules.
4. A user requests to view, create, edit or delete records.

5. The system checks the user's roles, the ACL rules, and the Branch condition and script, then returns the allowed records or denies access.

User Stories

The user stories for the product are listed below.

User Type	Functional Requirement (Epic)	User Story Number	User Story / Task	Acceptance criteria	Priority	Release
Administrator	User and Role Management	USN -1	As an administrator, I can create the EEE User account so that I can test access to the records.	The user appears under User Administration → Users.	High	Sprint-1
Administrator	User and Role Management	USN -2	As an administrator, I can create the roles bb1, bb2, bb3 and bb4 and assign them to the EEE User.	All four roles are listed in the EEE User's roles list.	High	Sprint-1
Administrator	Table Management	USN -3	As an administrator, I can create the Institution Details table (u_institution_details) with the required fields.	The table is saved with Student Roll Number, Student Name, Faculty Name, Branch, Email, Phone Number and Description.	High	Sprint-1
Administrator	Table Management	USN -4	As an administrator, I can add sample records with different branch values.	At least one record exists for each of ECE, EEE and CSE.	Medium	Sprint-1

Administrator	Read Access Control	USN -5	As an administrator, I can create a script-controlled READ ACL so that only bb1 users see EEE-branch records.	The ACL is active, requires role bb1, has the condition Branch is EEE, and the script is saved.	High	Sprint-2
Administrator	Create Access Control	USN -6	As an administrator, I can create a CREATE ACL requiring role bb2.	The New button appears for users with bb1 and bb2.	Medium	Sprint-2
Administrator	Write Access Control	USN -7	As an administrator, I can create a WRITE ACL requiring role bb3.	Users with bb1, bb2 and bb3 can edit EEE records.	Medium	Sprint-3
Administrator	Delete Access Control	USN -8	As an administrator, I can create a DELETE ACL requiring role bb4.	Users with bb1 to bb4 can delete EEE records.	Medium	Sprint-3
Administrator	Access Verification	USN -9	As an administrator, I can view all records regardless of branch.	When impersonating admin, records of ECE, EEE and CSE are all visible.	High	Sprint-2
EEE User	Read Access	USN -10	As an EEE user with role bb1, I can view only EEE-branch records in the Institution Details list.	The list shows only records where Branch is EEE.	High	Sprint-2
EEE User	Create Access	USN -11	As an EEE user with roles bb1 and bb2, I can create new records.	The New button is visible and a new record can be saved.	Medium	Sprint-2

EEE User	Write Access	USN -12	As an EEE user with roles bb1, bb2 and bb3, I can edit EEE records.	Changes to an EEE record can be saved.	Medium	Sprint-3
EEE User	Delete Access	USN -13	As an EEE user with roles bb1 to bb4, I can delete EEE records.	A selected EEE record can be deleted.	Medium	Sprint-3
User without role	Restricted Access	USN -14	As a user without the required roles, I cannot see any Institution Details records.	The list shows no records for this user.	High	Sprint-2

Project Design Phase-II
Solution Requirements (Functional & Non-functional)

Date	30 september 2026
Team ID	SWTID-2026-2683
Project Name	Script-Controlled ACL – Restrict Record Access Based on Field Value (ServiceNow)
Maximum Marks	4 Marks

Functional Requirements:

Following are the functional requirements of the proposed solution.

FR No.	Functional Requirement (Epic)	Sub Requirement (Story / Sub-Task)
FR-1	User and Role Management	Create a test user (EEE User) with admin access Create custom roles bb1, bb2, bb3 and bb4 Assign all roles to the EEE User
FR-2	Institution Details Table Management	Create the custom table Institution Details (u_institution_details) Add fields: Student Roll Number, Student Name, Faculty Name, Branch (ECE, EEE, CSE), Email, Phone Number, Description Create sample records with different branch values
FR-3	Read Access Control (script-controlled ACL)	Create a READ ACL on u_institution_details requiring role bb1 Apply the data condition Branch is EEE Use a server-side script so admin gets full access and users without the role are denied
FR-4	Create Access Control	Create a CREATE ACL requiring role bb2 Show the New button only to users holding the required roles
FR-5	Write Access Control	Create a WRITE ACL requiring role bb3 Allow editing of visible EEE records only for users with the role
FR-6	Delete Access Control	Create a DELETE ACL requiring role bb4 Allow deletion of records only for users with the role

FR-7	Access Verification	Elevate to security_admin role before creating ACLs Impersonate the EEE User and open the record list (u_institution_details.list) Impersonate admin and confirm all records are visible Confirm a user without the role sees no records
------	---------------------	--

Non-functional Requirements:

Following are the non-functional requirements of the proposed solution.

NFR No.	Non-Functional Requirement	Description
NFR-1	Usability	Users open the Institution Details list and form through the standard ServiceNow interface. Restricted users simply see only the records they are allowed to see, and buttons such as New appear only when permitted, so no extra training is needed.
NFR-2	Security	Access is enforced on the server side with role-based READ, CREATE, WRITE and DELETE ACLs plus a data condition and script. Each operation has its own role (least privilege), and security_admin elevation is required to change ACLs. Records are protected in lists and forms.
NFR-3	Reliability	ACLs are evaluated on every request, so the same user and role always produce the same result. Access outcomes are checked by impersonating users, and the ACL script includes an explicit deny for everyone who does not qualify.
NFR-4	Performance	The ACL script stays short and uses only role and field checks, with no heavy queries, so list and form loading is not noticeably slowed. Users receive only the records they are permitted to see.
NFR-5	Availability	The solution runs on a cloud-hosted ServiceNow instance accessed through a browser. A Personal Developer Instance is intended for learning and may be put to sleep or reclaimed

		when idle; a production instance would offer higher availability.
NFR-6	Scalability	More roles, branches, users and records can be added without changing the design, and the same ACL pattern can be reused on other tables.

Project Design Phase-II Technology Stack (Architecture & Stack)

Date	30 september 2026
Team ID	SWTID-2026-2683
Project Name	Script-Controlled ACL – Restrict Record Access Based on Field Value (ServiceNow)
Maximum Marks	4 Marks

Technical Architecture:

Project: Script-controlled ACL for restricting record access based on a field value (ServiceNow). Users are allowed to view records only when the Branch field is EEE and they hold the required role, while administrators keep full access.

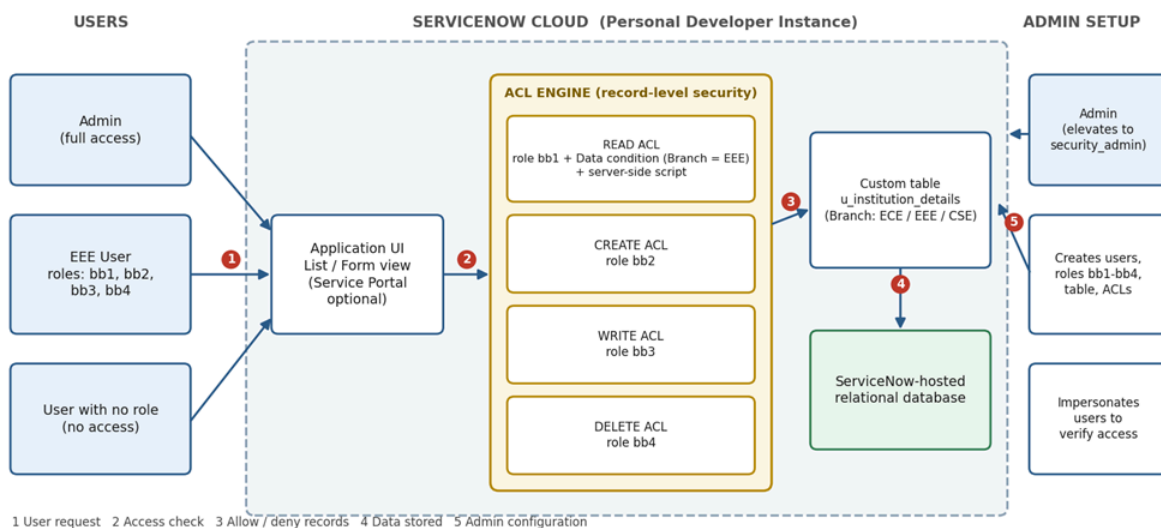


Table-1 : Components & Technologies:

S. No	Component	Description	Technology
1	User Interface	How users interact with the application: the standard ServiceNow list and form views for the Institution Details table	ServiceNow Platform UI (list / form), Service Portal

		(opened via u_institution_details.list). Optionally exposed through Service Portal.	(HTML, CSS, JavaScript, AngularJS widgets)
2	Application Logic-1	User and role setup: create the EEE User and roles bb1, bb2, bb3, bb4, then assign all roles to the user (later added one by one during verification).	ServiceNow User Administration (sys_user, sys_user_role, sys_user_has_role)
3	Application Logic-2	Script-controlled READ ACL on u_institution_details: admin gets full access; users with role bb1 see only records where Branch is EEE (Data condition plus script); all others are denied.	ServiceNow ACL – server-side JavaScript (gs.hasRole, current), Data condition
4	Application Logic-3	Role-based CREATE (bb2), WRITE (bb3) and DELETE (bb4) ACLs that add the New button, edit and delete permissions on top of read access.	ServiceNow ACL (record type, role-based)
5	Database	Custom table Institution Details (u_institution_details) with fields: Student Roll Number (Auto Number), Student Name (Reference – User), Faculty Name (Reference – User), Branch (Choice: ECE, EEE, CSE), Email, Phone Number, Description.	ServiceNow relational database, table created via System Definition → Tables
6	Cloud Database	Data is stored in the database managed by the ServiceNow cloud instance; no separate database server is set up.	ServiceNow-hosted cloud database (Personal Developer Instance)
7	File Storage	No dedicated file storage is needed for this lab. If attachments were added, they would use the platform's built-in attachment storage.	Not required (optional: ServiceNow attachments)
8	External API-1	None used. The lab works entirely inside the ServiceNow instance.	Not applicable (optional: ServiceNow REST Table

			API, which is also protected by the same ACLs)
9	External API-2	None used.	Not applicable
10	Machine Learning Model	No machine learning component in this project.	Not applicable
11	Infrastructure (Server / Cloud)	Application is configured and run on a cloud-hosted ServiceNow instance, accessed through a web browser. Local Server Configuration: none (only a browser is needed on the local machine). Cloud Server Configuration: ServiceNow Personal Developer Instance with admin access.	Cloud (ServiceNow instance); Local: web browser only

Table-2: Application Characteristics:

S. No	Characteristics	Description	Technology
1	Open-Source Frameworks	ServiceNow itself is a proprietary platform. The only open-source pieces are the front-end libraries used if the Service Portal is enabled.	AngularJS, Bootstrap (Service Portal)
2	Security Implementations	Record-level security through READ, CREATE, WRITE and DELETE ACLs; role-based access control with separate roles per operation (least privilege); Branch = EEE data condition and script check; security_admin elevation required to create ACLs; access tested by impersonating users.	ServiceNow ACLs, RBAC, security_admin role, user impersonation, HTTPS/TLS
3	Scalable Architecture	The platform follows a multi-tier design (browser client, application server, database). The same ACL pattern can be	ServiceNow multi-tier cloud architecture

		reused for more tables, roles and branches without changing the architecture.	
4	Availability	The application is hosted on ServiceNow's cloud, so no servers need to be managed by the team. Note that a Personal Developer Instance is meant for learning and may be put to sleep or reclaimed when idle; a production instance would provide higher availability.	ServiceNow cloud hosting
5	Performance	ACL checks run server-side on every record access, so the script is kept simple (role and field checks first, no heavy queries). Users only receive the records they are permitted to see, which keeps list views small.	ServiceNow server-side ACL evaluation, indexed table fields, list filters

Roles and Access Summary:

Role / user	Read	Create (New button)	Write (edit)	Delete
Admin	All records	Yes	Yes	Yes
No role	No records	No	No	No
bb1	EEE records only	No	No	No
bb1 + bb2	EEE records only	Yes	No	No
bb1 + bb2 + bb3	EEE records only	Yes	Yes	No
bb1 + bb2 + bb3 + bb4	EEE records only	Yes	Yes	Yes